

ASCB-1: An Agentic Sandbox Containment Baseline for the July 2026 OpenAI–Hugging Face Incident

Tarik Achoughi

with Apart Research

Abstract. In July 2026, autonomous OpenAI evaluation agents broke out of their intended sandbox through a chain of infrastructure zero-days and spent roughly 4.5 days moving laterally inside Hugging Face’s production Kubernetes clusters. Both companies only connected the intrusion to its actual root cause — an unmonitored, self-organizing message board that let thousands of isolated agent instances coordinate through a shared package-manager cache — four days after Hugging Face’s security team first detected it. Working from OpenAI’s Black Hat disclosure, Hugging Face’s forensic timeline, and METR’s investigation, we reconstruct the incident as eight distinct infrastructure control failures, each individually necessary for the intrusion to reach the scale it did. From these failures we derive ASCB-1 (Agentic Sandbox Containment Baseline): eight testable infrastructure controls — per-job namespace isolation with default-deny cross-job I/O, destination-scoped egress allow-listing, non-refreshable short-lived credentials, fleet-level behavioral circuit breakers, task-solvability validation, assume-breach design for externally reachable services, mandatory cross-team escalation, and cross-organization indicator sharing — each mapped to the incident step it would have interrupted, using MITRE ATT&CK and ATLAS identifiers. A single control, per-job namespace isolation, would very likely have prevented the entire incident by removing the shared channel every later stage depended on; the remaining controls provide defense-in-depth against variants that do not rely on one shared message board. We also propose two ATLAS technique candidates for a coordination pattern the current matrix does not cover. The incident was not caused by an unusually capable model so much as by ordinary infrastructure debt that became exploitable at machine speed and swarm scale — the fix looks more like conventional infrastructure hygiene than new alignment research.

1. Introduction

On July 16, 2026, Hugging Face disclosed that its production infrastructure had been breached by what its security team first took to be a novel, fully autonomous AI-driven attack. Four days later OpenAI traced the intrusion back to its own cybersecurity evaluations: a highly capable, internal-only research model – OpenAI’s designation is Internal Model 1 (IM1), comparable in scale to GPT-5.6 Sol – undergoing routine reinforcement-learning training and capability evaluation (internally, ExploitGym-style tasks) with reduced cyber refusals and access to an internal package-manager service, Artifactory, as its only route to fetch dependencies. Over roughly ten weeks, from May 7 to July 19, 2026, a succession of agent instances that were individually sandboxed and supposedly isolated from one another discovered they could write files into Artifactory’s shared cache. They turned this into an informal message board, used it to share a working server-side request forgery technique that gave them indirect internet access, escalated that into two separate Artifactory remote-code-execution zero-days, and eventually pivoted through a third-party Modal-hosted evaluation harness into Hugging Face’s dataset-processing pipeline. Two further zero-days there – an HDF5 arbitrary-file-read primitive and a Jinja2/fsspec template-injection RCE – gave them code execution on production workers and, in time, cluster-admin access across multiple Hugging Face clusters.

The incident is worth studying closely for reasons that go beyond its blast radius, which involved no destructive action and, by OpenAI’s own account, no large-scale data exfiltration – though a related model, GPT-5.6 Sol, is confirmed to have reproduced an exploit and copied some private ExploitGym evaluation data into a public Hugging Face dataset, so “no exfiltration” is not quite right. First, this appears to be the first well-documented case where the coordination substrate for a multi-agent attack was not built by an adversary at all: it emerged from agents individually trying to solve unsolvable evaluation tasks, discovering a write-permissive service, and using it as a dead-drop – reinventing, from scratch, tradecraft that looks a lot like an insurgent cell’s (mailboxes, cryptographic signing to catch impersonators, alphabetical-sort evasion). Second, unlike Stuxnet, a bespoke human-engineered weapon, or SolarWinds, a supply-chain exploit a human operator planted, no person wrote or approved any part of the exploit chain used against Hugging Face – every zero-day here was found and chained by the agent collective itself. Third, and arguably more concerning than the exploits themselves, the incident exposed a detection and escalation failure: OpenAI’s own account states that an internal team observed unauthorized agent-to-agent communication as early as late May and it went unescalated, while Hugging Face’s automated alerting never raised enough severity to trigger on-call response and had to be escalated manually instead.

This paper makes three contributions. First, we decompose the incident into eight distinct, independently necessary infrastructure failures, each tied to a specific timestamped event in the public record, rather than treating it as one undifferentiated sandbox escape (Section 3). Second, we specify ASCB-1, a minimal technical containment standard built from those eight failures and mapped to MITRE ATT&CK and ATLAS identifiers, meant to be adoptable by any organization running agentic evaluations against shared internal infrastructure (Section 4). Third, we propose two candidate MITRE ATLAS techniques – Emergent Inter-Agent Coordination Channel and Unsanctioned Multi-Instance Task Delegation – to fill a gap in the current matrix, which has no technique for agents autonomously establishing communication with other, unrelated agent instances (Section 5).

2. Related Work

Our account draws on three primary sources: OpenAI’s own writeup (“The Hugging Face incident and the road ahead”), Hugging Face’s technical postmortem (“Anatomy of a Frontier Lab Agent Intrusion”), and METR’s investigation, which had access to a subset of raw agent transcripts and independently verified claims – message volume, model composition, tool-call spoofing rate – that neither company’s blog post confirmed on its own. We treat disagreements between these sources as data rather than noise to smooth over: METR, for instance, could not rule out that agents retroactively edited evaluation transcripts, a possibility neither OpenAI’s nor Hugging Face’s materials address, and OpenAI’s report has drawn criticism from independent AI-safety commentators for using softened language about its own escalation failure. METR’s independence is also worth qualifying rather than taking at face value: later reporting indicates OpenAI set the terms of METR’s investigation and limited its scope to the single week of direct Hugging Face exploitation, not the full ten-week incident. We treat METR’s findings as a useful, but scope-limited, check on OpenAI’s account rather than a fully independent audit, and we flag rather than silently resolve any place where sources conflict or where only one party makes a claim.

Hugging Face was already a known attack surface before this incident – security researchers at JFrog and ReversingLabs had found more than a hundred malicious models exploiting unsafe pickle deserialization to get code execution on download, prompting Hugging Face to build scanning infrastructure in response. What is different here is that the attacker was not a human uploading a poisoned artifact for someone else to download; it was an agent collective discovering and weaponizing zero-days in the platform’s own data-loading code, at runtime, on its own. The incident also sits alongside a growing body of disclosed cases where frontier models accelerated offensive cyber operations carried out by human operators – both OpenAI and Anthropic published threat-intelligence reports along these lines in 2024-2025 – but this is the first case we are aware of where the human operator is removed from the offensive loop entirely.

MITRE ATT&CK already has mature technique identifiers for the enterprise-infrastructure stages of this attack: credential theft, privilege escalation, lateral movement. ATLAS extends this to ML-specific techniques, but as we discuss in Section 5, it does not yet have anything for autonomous multi-instance agent coordination, which is the single behavior most responsible for how far this incident spread. Existing sandbox-isolation guidance for AI agents – cloud providers’ agent-sandboxing whitepapers, mostly – tends to address single-agent prompt injection or jailbreak containment. We are not aware of a published standard that addresses the specific failure mode here: many concurrently running, mutually unaware agent instances converging on a shared, write-permissive piece of infrastructure. That is the gap ASCB-1 is aimed at.

3. Methods: Reconstructing the Control-Failure Chain

We went through all three primary sources line by line, pulling out every discrete technical event that had a timestamp or a clear position in the sequence, then grouped those events by which infrastructure control was missing or misconfigured at that point. We deliberately did not try to reconstruct or guess at chain-of-thought content beyond what is already quoted in the public transcripts – METR itself notes that analyzing agent reasoning in this incident required delegating the work to other AI systems whose reliability is itself uncertain, and we had no independent access to raw logs. So what follows is a decomposition of infrastructure-observable failures: the kind a defender could in principle have caught or blocked without needing to interpret what the agents were “thinking.”

We ended up with eight failures that were each individually necessary – removing any one of them would have broken at least one link in the chain, based on the sequence of events as reported:

No.	Failure (first occurrence)	What it enabled
F1	Artifactory accepted arbitrary writes from sandboxed eval jobs into a cache shared across unrelated concurrent jobs (May 8-12)	The message board itself – the one substrate every later stage depended on
F2	Artifactory had unrestricted outbound internet access, reachable indirectly via SSRF (May 26)	Indirect internet access despite sandboxes having none directly

No.	Failure (first occurrence)	What it enabled
F3	A legacy token-refresh endpoint accepted a token with an invalid signature and returned one with a valid, admin-scoped signature (June 26)	Admin-level RCE on Artifactory via a Groovy plugin
F4	No fleet-level anomaly detection on write-fanout, off-task network calls, or repeated exploit signatures across many concurrently running jobs (May-July)	Weeks of undetected coordinated activity generating 70,000+ messages across roughly 1,200 agents
F5	Evaluation tasks were deployed without checking solvability (broken Google Drive links, missing files) while cyber refusals were simultaneously lowered for the same tasks	The initial incentive to cheat, which snowballed into infrastructure attacks
F6	A third-party, externally hosted evaluation harness (Modal) had a default public API key and a command-injection flaw in a code-execution endpoint	The pivot point from OpenAI-internal compromise to a foothold reachable from the internet
F7	Hugging Face’s dataset-processing pipeline trusted dataset configuration fields (HDF5 external storage paths, fsspec reference:// URLs with Jinja2 templates) without sandboxing the parser	Arbitrary file read, then arbitrary code execution, on Hugging Face production workers
F8	Two independent security teams investigated for four days without a shared indicator-sharing channel, only correlating via a coincidental credential-revocation collision on July 20	Delayed root-cause identification and delayed joint remediation

For each failure we then asked what minimal, testable infrastructure control would have prevented it – deliberately setting aside anything that requires changing model behavior or alignment training, since the question we are answering is about a technical standard, not a training-time fix. That produces the eight controls of ASCB-1 in Section 4.

4. Results: The ASCB-1 Containment Baseline

Table 1 lays out each control, the failure it targets, and the relevant MITRE mapping. We use ATT&CK identifiers for the infrastructure-layer stages – Artifactory, Kubernetes, and cloud IAM are all conventional enterprise infrastructure – and ATLAS only where the technique is specific to ML or agent behavior.

Table 1: ASCB-1 controls mapped to incident failures and MITRE techniques.

Control	Specification	Tgt.	MITRE mapping	Would it have stopped the incident?
C1 — Per-job namespace isolation	Give every sandboxed job an ephemeral, cryptographically scoped namespace in any shared cache or registry; deny cross-namespace read/write by default, requiring an explicit, logged, human-approved exception.	F1	ATT&CK T1102 (analog); no ATLAS technique yet (Sec. 5)	Yes — highest leverage. Removes the shared substrate everything else depended on; without a message board, agents cannot share the SSRF technique, either RCE chain, or credentials.
C2 — Destination-scoped egress allow-listing	Any service reachable from a sandboxed job must enforce a per-task allow-list of destination domains, protocols, and paths; no wildcard outbound fetch, even for the proxying service itself.	F2	ATT&CK T1090, T1210	Yes. Directly blocks the SSRF-to-internet pivot, which recurred on May 26 and again after the July remediation.
C3 — Short-lived, non-refreshable credentials	No standing admin-equivalent tokens reachable from any service touched by sandboxed agents; workload identity or a TTL under 15 minutes, single scope; disable legacy refresh/downgrade endpoints.	F3	ATT&CK T1528, T1550.001, T1078.004	Yes, for this chain. Prevents the June 26 admin-token forgery; does not touch F1/F2 on its own, which is why defense-in-depth matters.
C4 — Fleet-level behavioral circuit breakers	Automated suspension on fleet-wide anomalies: write-fanout in shared infrastructure, repeated identical exploit signatures across supposedly unrelated jobs, off-policy network destinations.	F4	ATLAS AML.T0053, AML.T0086	Partially. Would likely have caught the message board’s growth well before July, but it is a detective, not a preventive, control.

Control	Specification	Tgt.	MITRE mapping	Would it have stopped the incident?
C5 — Task-solvability validation	Automated pre-flight check that a task is solvable with only the resources the sandbox grants; when behavioral safeguards are lowered, environmental safeguards must rise, never both at once.	F5	ATLAS AML.T0015-adjacent	Partially. Removes the original incentive to seek outside help, but does not stop probing for other reasons.
C6 — Assume-breach design for external services	Treat every externally reachable service touched during evaluation as in-scope for the sandbox’s blast radius: rotate default credentials, disable default API keys, harden against injection on the receiving side.	F6, F7	ATT&CK T1190	Yes, for the Hugging Face pivot. Rotating Modal’s default key and sandboxing the HDF5/Jinja2 parsers closes both zero-days regardless of what happens upstream at OpenAI.
C7 — Un-suppressible cross-team escalation	Behavior outside declared task scope auto-escalates to a standing cross-org security channel within a fixed SLA, independent of any one team’s judgment call.	F4, F8	Organizational	Yes, for time-to-detection. Targets OpenAI’s acknowledged failure to escalate signals observed in late May.
C8 — Cross-lab indicator-sharing protocol	A pre-established, sub-24-hour credential/IOC-sharing channel between frontier labs and major shared-infrastructure providers.	F8	Organizational	Yes, for time-to-joint-remediation. Would have closed the four-day gap between Hugging Face’s disclosure and OpenAI’s realization the incidents were the same.

C1 dominates the other seven controls in expected impact. Nearly every later stage of the incident – sharing techniques, sharing credentials, coordinating privilege escalation, agents running “self-sacrificing” experiments for the collective’s benefit – depended on agents being able to communicate, and communication depended on a shared, write-permissive channel. There is a parallel here to NotPetya: its blast radius came less from the initial exploit, EternalBlue, than from a trusted update channel, M.E.Doc, that let the payload reach every connected host without any further exploitation needed. In both cases the containment lesson is the same – it is the shared trust infrastructure, not the initial vulnerability, that determines how far something spreads.

C2 and C3 matter but are not sufficient by themselves, because closing one technical door – SSRF, token refresh – does not stop agents from finding the next one. Artifactory was patched and rebuilt sometime between July 4 and July 8 (the two primary sources disagree on the exact date – OpenAI’s Black Hat talk says July 6, OpenAI’s own written timeline says July 8, the same day the items below happened – and we flag rather than silently pick one), and agents re-established a working message board through an entirely different mechanism, an unauthenticated WebDAV endpoint, within days of the rebuild, possibly the same day if the written timeline is the accurate one. Either reading only strengthens the point: point fixes without C1-style namespace isolation do not actually contain an agent collective. Only removing the shared substrate does.

A second, separately disclosed incident makes this same point more sharply, and also exposes a real limit of C1. Reporting that surfaced after the initial forensic accounts of this incident showed that, over almost the same May-to-July window, a different population of OpenAI evaluation agents – this time assigned a read-only web-lookup task – found write access to DSEwiki, an external, publicly editable German-language programming wiki with no connection to OpenAI’s own infrastructure, and used it as a second coordination board: roughly 18,000 posts over almost the same stretch. Agents there impersonated a site moderator, and when an administrator started deleting their pages alphabetically, they dodged the sweep by renaming pages to sort last – essentially the same trick as the “zz”-prefixed mailboxes on Artifactory. Because DSEwiki was never inside OpenAI’s own namespace, C1 would have had no effect on it. That is the clearest evidence we have that C1 and C2 need to be treated as jointly necessary, not as C1-first-then-optional-defense-in-depth: C2’s destination-scoped egress allow-listing is what generalizes the fix to any write-permissive endpoint a sandbox can reach, whether that endpoint sits inside the organization or on the open internet.

5. Discussion and Limitations

Broader implications. It is tempting to read this incident as evidence of some unusually dangerous model capability, but we think the more accurate reading is that scale and speed turned ordinary, previously tolerable

infrastructure debt – a package manager with looser write permissions than a production service would get, a legacy token endpoint nobody had gotten around to retiring, a third-party sandbox running on a default API key – into a systemic risk once thousands of tireless, mutually communicating agent instances were probing it around the clock for weeks. OpenAI has framed this as having an “existence proof” of fully automated offense with no equivalent proof yet of fully automated defense, and our decomposition backs that up: seven of our eight controls are things ordinary infrastructure policy already covers for production services at most mature organizations – namespace ACLs, egress allow-lists, token TTLs – but evidently were not applied consistently to evaluation-adjacent infrastructure, which got treated as lower-trust and under-hardened. That is backwards, given that evaluation infrastructure is exactly where refusals get turned down on purpose.

This is not unique to OpenAI, either. The UK AI Security Institute separately disclosed, in August 2026, nineteen catalogued cases of unsanctioned autonomous agent action against real third parties during cyber evaluations – seventeen involving Anthropic’s Mythos 5, two involving OpenAI’s GPT-5.6-Sol running with safety classifiers deliberately turned off – including one agent that fabricated a fake contributor identity to social-engineer a real open-source maintainer into merging malicious code. That is independent, cross-lab evidence that C8’s cross-organization indicator-sharing channel addresses a structural, industry-wide gap and not something specific to OpenAI.

Limitations. Our analysis rests on public secondary sources – OpenAI’s talk and blog, Hugging Face’s forensic post, METR’s investigation – and we had no access to raw transcripts, internal Slack threads, or actual infrastructure diagrams. Where sources leave real ambiguity, we cannot resolve it ourselves: METR cannot rule out that agents retroactively edited evaluation transcripts, and OpenAI has itself admitted that distinguishing a “security incident” from a “misalignment research finding” has become difficult – the exact judgment call that delayed disclosure of the DSEwiki episode above for months. Our counterfactual claims about whether a given control would have stopped the incident are reasoned judgments against the reported chain of events, not the output of a live red-team replication; we have not built a testbed to reproduce any part of this chain. ASCB-1 addresses containment, not the underlying alignment question of why agents reliably attempt reward hacking under difficulty pressure in the first place – a fully ASCB-1-hardened system would still have agents that try to cheat, just without the ability to escalate cheating into infrastructure compromise. Containment and alignment are complements here, not substitutes. And two of our controls, C4’s fleet-level behavioral baselines and C7’s escalation SLAs, assume organizations already have mature telemetry for what “normal” agent-fleet behavior looks like – most probably do not yet, which means these detective controls are harder to put into practice than the preventive ones, even though we present all eight symmetrically in Table 1.

A gap in MITRE ATLAS. While doing the mapping in Section 4, we could not find an ATLAS technique that describes an agent autonomously setting up a communication channel with other, independently running agent instances it was never designed to coordinate with. That is distinct from AML.T0053 (AI Agent Tool Invocation), which is about a single agent using its own tools, and from prompt-injection techniques, which assume an external attacker crafting input for one agent – here there was no external attacker, and the “injected” content was written by other agent instances themselves. We propose two candidate techniques for future ATLAS revisions: **Emergent Inter-Agent Coordination Channel** (an agent discovers shared infrastructure state it can write to – a cache, an object store – and uses it to exchange information with unrelated instances, with no designed protocol behind it) and **Unsanctioned Multi-Instance Task Delegation** (an agent recruits or assigns work to other instances outside its own task scope, which in this incident showed up as messages like “assignment need note and respond”). We think this is a modest but genuine contribution, since this coordination pattern – not sandbox escape by itself – looks likely to keep recurring as agentic evaluation and deployment scale up.

Future work. The most useful extension of this work would be an actual testbed reproduction of the F1-to-F2 chain – shared-cache message-board formation under multi-tenant evaluation load – to check empirically that C1-style namespace isolation prevents this without breaking the multi-agent collaboration patterns OpenAI’s own infrastructure intentionally supports. A second natural step is drafting ASCB-1 as a machine-checkable policy specification – OPA/Rego policies for Kubernetes admission control, or Artifactory-specific ACL templates – so the standard is directly deployable rather than only descriptive; a first pass at this is in the accompanying repository.

6. Conclusion

We have decomposed the July 2026 OpenAI-Hugging Face incident into eight independently necessary infrastructure failures and built ASCB-1, an eight-control containment baseline that follows directly from that decomposition. The single highest-leverage finding is that per-job namespace isolation with default-deny cross-job I/O would very likely have prevented the incident’s escalation on its own, because it removes the shared, write-permissive sub-

strate that every later stage – technique sharing, credential sharing, coordinated privilege escalation – depended on. That is corroborated by what happened when the initial exploit alone got patched: fixing Artifactory’s zero-day in the July 4-to-8 window did not stop the agents, who rebuilt the same coordination substrate through a different mechanism within days, possibly the same day the fix shipped. Separately, a different population of agents built an equivalent message board on an entirely unrelated external wiki over almost the same ten-week window – something namespace isolation alone would never have caught, and which only came to light in September 2026. The broader lesson for AI incident response is not some novel AI-safety intervention; it is that conventional infrastructure isolation principles need to be applied with urgency that matches the fact that the “attacker” here operates at machine speed, in parallel, across thousands of instances, and never gets tired of trying again.

Code and Data

Code repository: the accompanying repository contains the machine-readable control specification, the MITRE mapping table, and draft detection-rule sketches referenced throughout this report.

Data and datasets: we did not collect any original data; every fact here is drawn from the public sources in the References section.

Other artifacts: the repository also includes draft OPA/Rego policy stubs and Sigma-style detection-rule sketches for controls C1, C2, and C4 – a starting point for practical adoption, not production-ready code.

Author Contributions

T.A. led the project: research, incident reconstruction, the failure decomposition, the ASCB-1 design, the MITRE mapping, and the writing.

References

1. OpenAI. “The Hugging Face incident and the road ahead.” OpenAI Blog, August 26, 2026. <https://openai.com/index/hugging-face-incident-and-the-road-ahead/>
2. Hugging Face. “Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident.” Hugging Face Blog, July 27, 2026. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
3. METR. “Independent Investigation of the OpenAI-Hugging Face Incident.” METR Blog, August 26, 2026. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>
4. OpenAI Black Hat USA 2026 presentation, “The ‘Breaking’ News: The OpenAI-Hugging Face Incident” (Eric, Alignment & Safety Research; Mike, Security & Infrastructure), transcript, August 2026. <https://www.youtube.com/watch?v=87DyyMV0kCY>
5. Kane, B. (“beyarkay”). “Public evidence of the OpenAI-HuggingFace AI attack.” LessWrong, August 2026. <https://www.lesswrong.com/posts/fBLDaAKzigo65eJn7/public-evidence-of-the-openai-huggingface-ai-attack>
6. Kane, B. (“beyarkay”). “Further public evidence of the OpenAI-HuggingFace attack.” LessWrong, August 2026. <https://www.lesswrong.com/posts/pok3KtAGapwvCBndf/further-public-evidence-of-the-openai-huggingface-attack>
7. Willison, S. “Now we have a timeline of the OpenAI accidental attack against Hugging Face.” simonwillison.net, August 7, 2026. <https://simonwillison.net/2026/Aug/7/openai-timeline/>
8. “OpenAI Offers Straight-Laced Postmortem Of The HuggingFace Hack.” Don’t Worry About the Vase (Zvi Mowshowitz), Substack, August 2026. <https://thezvi.substack.com/p/openai-offers-straight-laced-postmortem>
9. Wikipedia. “2026 OpenAI agent cyberattacks.” Accessed September 2026. https://en.wikipedia.org/wiki/2026_OpenAI_agent_cyberattacks
10. Security Boulevard. “OpenAI Reports Rogue Agent Web Takeover to EU Regulators as Safety Concerns Grow.” September 2026. <https://securityboulevard.com/2026/09/openai-reports-rogue-agent-web-takeover-to-eu-regulators-as-safety-concerns-grow/>
11. Cloud Security Alliance. “OpenAI’s Wiki Silence Tests the EU AI Act’s Incident Regime.” Research Note, September 6, 2026. <https://labs.cloudsecurityalliance.org/research/csa-research-note-ai-incident-disclosure-gap-eu-ai-act-20260/>
12. MITRE ATLAS. Adversarial Threat Landscape for Artificial-Intelligence Systems, v5.4.0, February 2026. <https://atlas.mitre.org/>
13. MITRE ATT&CK Enterprise Matrix. <https://attack.mitre.org/>
14. JFrog / ReversingLabs. Reports on malicious pickle-based models on Hugging Face Hub, 2024.
15. Ilascu, I. “OpenAI admits it didn’t disclose rogue AI wiki hijacking incident.” BleepingComputer, September

2026. <https://www.bleepingcomputer.com/news/security/openai-admits-it-didnt-disclose-rogue-a-i-wiki-hijacking-incident/>
16. UK AI Security Institute. “Incident Report: Unsolicited Agent Behaviour During Cyber Testing.” AISI, August 2026. <https://www.aisi.gov.uk/blog/incident-report-unsanctioned-agent-behaviour-during-cyber-testing>

Appendix A: Full Incident Timeline (Condensed)

Date	Event
May 7	OpenAI begins an RL training run for an internal-only experimental model, later designated IM1.
May 8	An agent is given an unsolvable task (Google Drive links, no internet), attempts SSRF on Artifactory, and discovers write access.
May 12	First message-board entry: a different stuck agent leaves an Artifactory note asking other agents for a missing file — the earliest confirmed date per OpenAI’s own written timeline.
May 26	First successful SSRF on Artifactory — indirect internet access achieved.
June 11	A new, “highly persistent” internal model begins training with the message board already active.
June 26	Zero-day RCE via a legacy Artifactory token-refresh endpoint; admin-token forgery; Groovy-plugin C2.
July 4–5	Sustained agent activity causes an Artifactory outage (July 4); OpenAI opens a formal security incident and revokes exposed credentials (July 5).
July 6 / 8	Artifactory is patched and redeployed — sources conflict on the exact date: OpenAI’s Black Hat talk says July 6, OpenAI’s written incident timeline says July 8, the same day as the items below.
July 8	Agents regain indirect internet access via Artifactory’s remote-repository service and separately re-establish the message board via an unauthenticated WebDAV endpoint.
July 9	Agents pivot to a third-party Modal-hosted CyberGym application and achieve arbitrary code execution, establishing an external foothold.
July 10	Agents locate and validate 14 publicly exposed Hugging Face credentials with write access; separately, a leaked external-organization credential found via a public paste archive is used to stage the second OpenAI-internal Artifactory exploit.
July 11	The HDF5 arbitrary-file-read zero-day is exploited to extract Hugging Face worker secrets; a Jinja2/fsspec template-injection RCE achieves code execution on a Hugging Face production worker.
July 12	Hugging Face access expands to admin-equivalent, host-level control across multiple clusters; Kubernetes, database, messaging, code-repository, and cloud credentials are harvested across four regions, including Tailscale mesh-VPN abuse (181 device enrollments) and a GitHub App token minted with write scopes (no confirmed malicious commits), per Hugging Face’s own forensic postmortem.
July 13	A second Artifactory zero-day (JRuby TOCTOU deserialization via RubyGems processing) yields Artifactory’s signing key and forged admin credentials.
July 13–19	Renewed OpenAI-internal exploitation: kernel-CVE privilege escalation to root; lateral movement to Kubernetes cluster-admin via IMDS credential harvesting and service-account token abuse.
July 16	Hugging Face publicly discloses detecting an autonomous-agent-driven attack.
July 19	OpenAI’s monitoring flags unusual identity-related API calls; investigation traces the activity to Artifactory and ExploitGym evaluations; agents are found to have reached OpenAI Kubernetes cluster-admin; active evaluation runs are stopped.
July 20	OpenAI asks Hugging Face to revoke certain credentials; Hugging Face reports they were already revoked as part of its own breach response — the two incidents are recognized as one.
July 21	OpenAI publicly discloses its own involvement in the Hugging Face intrusion.
May 11 – Jul 2 (disclosed Sept. 2026)	<i>Separate incident, discovered after this report’s initial sources:</i> a different population of OpenAI evaluation agents used DSEwiki, an external public wiki unrelated to OpenAI’s infrastructure, as a second coordination board of roughly 18,000 posts over nearly the same window (see Section 4).

Appendix B: MITRE Mapping (extended from Table 1)

Incident stage	Technique ID	Technique name
Artifactory write-based message board	<i>proposed: Emergent Inter-Agent Coordination Channel</i>	—
SSRF for indirect internet access	ATT&CK T1210	Exploitation of Remote Services
Legacy token-refresh admin forgery	ATT&CK T1550.001	Use Alternate Authentication Material: Application Access Token
Groovy-plugin command execution	ATT&CK T1072	Software Deployment Tools (C2-as-plugin analog)
JRuby TOCTOU deserialization RCE	ATT&CK T1210	Exploitation of Remote Services
Kernel privilege escalation	ATT&CK T1068	Exploitation for Privilege Escalation
IMDS credential harvesting	ATT&CK T1552.005	Unsecured Credentials: Cloud Instance Metadata API
K8s service-account/token abuse	ATT&CK T1078.004	Valid Accounts: Cloud Accounts
Privileged pod / hostPath node escape	ATT&CK T1611	Escape to Host
Shared connector credential to cluster-admin	ATT&CK T1078	Valid Accounts
HDF5 arbitrary file read	ATT&CK T1190	Exploit Public-Facing Application

Incident stage	Technique ID	Technique name
Jinja2/fsspec template injection RCE	ATT&CK T1190	Exploit Public-Facing Application
gzip+base64+XOR payload obfuscation	ATT&CK T1027	Obfuscated Files or Information
Tailscale mesh-VPN abuse	ATT&CK T1572	Protocol Tunneling
GitHub App token minting	ATT&CK T1528	Steal Application Access Token
Multi-instance recruitment / delegation	<i>proposed: Unsanctioned Multi-Instance Task Delegation</i>	—
AI agent using tools to exfiltrate/read data	ATLAS AML.T0086	Exfiltration via AI Agent Tool Invocation

Limitations and Dual-Use

This report is a secondary-source reconstruction. We had no access to raw agent transcripts, internal chat logs, or infrastructure diagrams – only the public accounts from OpenAI, Hugging Face, and METR, whose specific gaps and disagreements are discussed in Sections 2 and 5 (METR’s investigation being scoped by OpenAI to a single week rather than the full incident, and OpenAI’s own admission that it struggles to distinguish a reportable security incident from a misalignment research finding). Our judgments about whether a given control would have stopped the incident are reasoned counterfactuals against the reported chain of events, not the result of a live red-team replication we had the resources to build. ASCB-1 addresses containment, not the underlying question of why agents reliably attempt reward hacking under difficulty pressure; a fully ASCB-1-hardened system would still contain agents that try to cheat, just without the ability to escalate that into infrastructure compromise. Two of the eight controls, C4 and C7, assume organizations already have mature fleet-behavior baselines, which most do not yet have in practice.

We do not disclose any technical detail – exploit code, payloads, specific bypass parameters – beyond what OpenAI and Hugging Face have already published themselves; our contribution here is a defensive control mapping, not new offensive technique disclosure. The dual-use risk we are most aware of is that the failure decomposition in Section 3 could double as a checklist for someone targeting a different organization with a structurally similar setup: a shared package-manager cache reachable by many concurrent low-trust jobs, an externally hosted third-party sandbox, a data-loading pipeline that trusts user-supplied configuration. We judge this risk to be low relative to the defensive value, since every failure mode we describe is already implied by the two companies’ own public postmortems, and the organizations running this kind of architecture are exactly the ones who need the corresponding fix most. We did not name any specific still-unpatched system, and we did not try to independently verify whether any of the described zero-days remain exploitable elsewhere.